

W10D01 — Docker Compose Fundamentals

ML Engineer Journey | Phase 3 — Docker & Clean Architecture | 11 May 2026

1. Apa itu Docker Compose?

Docker Compose adalah tool untuk mendefinisikan dan menjalankan ekosistem multi-container dalam satu file deklaratif: **docker-compose.yml**. Sebelumnya, setiap container dijalankan manual satu per satu dengan perintah *docker run* yang panjang dan rentan typo. Dengan Compose, seluruh konfigurasi — ports, volumes, environment, network — ditulis sekali dan dijalankan dengan satu perintah.

Analogi: Bayangkan restoran yang setiap pagi harus menyalakan kompor, oven, dan mesin kasir secara manual satu per satu. Docker Compose adalah checklist yang menjalankan semuanya sekaligus, dalam urutan yang benar, dengan instruksi yang konsisten setiap hari.

2. docker run vs docker-compose — Perbandingan

Aspek	docker run	docker-compose up
Cara kerja	1 container, 1 perintah	Semua service, 1 perintah
Konfigurasi	Flag CLI panjang	File YAML deklaratif
Reproducibility	Mudah salah ketik	Konsisten, bisa di-commit
Multi-container	Manual, satu per satu	Otomatis + networking
Restart policy	Harus tambah flag	Dideklarasikan di file
Environment vars	--env atau --env-file	environment: atau env_file:

3. Anatomi docker-compose.yml

Berikut adalah struktur lengkap dengan penjelasan setiap key. Pahami setiap field — bukan sekadar template yang di-copas.

```
version: '3.9'           # Versi schema Compose – gunakan 3.9 sebagai default

services:                # Blok utama: definisi semua container
  risk-api:               # Nama service = hostname di internal Docker network
    build:
```

```

context: .          # Direktori build context (lokasi Dockerfile)
dockerfile: Dockerfile
container_name: risk-scoring-api
ports:
  - '8000:8000'     # HOST:CONTAINER — port mapping
environment:
  - APP_ENV=development
  - LOG_LEVEL=INFO
# env_file:
#   - .env           # Gunakan ini untuk credentials — jangan hardcode
volumes:
  - api-logs:/app/logs # Named volume untuk persist logs
restart: unless-stopped # Restart otomatis kecuali dihentikan manual
healthcheck:
  test: ['CMD', 'curl', '-f', 'http://localhost:8000/health']
  interval: 30s      # Cek setiap 30 detik
  timeout: 10s       # Tunggu max 10 detik per cek
  retries: 3         # 3x gagal = container unhealthy
  start_period: 40s  # Jangan cek di 40 detik pertama

volumes:             # Deklarasi named volumes
  api-logs:
    driver: local

```

4. Penjelasan Key yang Sering Disalahpahami

version

Bukan versi Docker Compose CLI yang terinstall. Ini adalah versi *schema* file YAML. Versi 3.x mendukung fitur modern: healthchecks, deploy constraints, dan configs. Gunakan **3.9** sebagai standar.

service name sebagai DNS hostname

Nama service di Compose (*api*, *db*, *redis*) otomatis menjadi hostname di internal Docker network. Service *api* bisa memanggil *db* dengan **http://db:5432** tanpa konfigurasi tambahan — karena Docker membuat default network untuk semua service dalam satu file Compose. Penting: dua file Compose yang berbeda = dua network terpisah = tidak bisa saling memanggil via nama.

ports: HOST:CONTAINER

Format "**9000:8000**" artinya akses dari luar lewat port 9000 di host, tapi di dalam container tetap port 8000. Berguna menghindari konflik port jika ada dua service yang sama-sama butuh port 8000 di dalam container.

restart policy

Policy	Perilaku	Kapan pakai
--------	----------	-------------

no	Tidak pernah restart	Development / one-off tasks
always	Selalu restart, termasuk setelah manual stop + manual restart	Service in production
unless-stopped	Restart otomatis kecuali dihentikan manual. Setelah di restart, tidak jadi jika sebelumnya di-stop	Default yang standar. Tidak jadi jika
on-failure	Hanya restart jika exit code non-zero	Batch jobs

5. Bind Mount vs Named Volume

Ini perbedaan kritis yang sering menjadi sumber bug di production.

	Bind Mount <code>./logs:/app/logs</code>	Named Volume <code>api-logs:/app/logs</code>
Dikelola oleh	Host (kamu)	Docker
Akses dari host	Langsung, real-time	Perlu docker volume inspect
Cocok untuk	Development, debug logs	Production, data persistence
Portability	Terikat path host	Portable antar environment
Jika path tidak ada	Docker buat otomatis (kosong)	Docker buat volume baru

Aturan praktis: Pakai bind mount saat kamu butuh lihat file langsung dari host (development, debug). Pakai named volume saat data harus persist dan portable (production database, model artifacts).

6. Perintah docker-compose yang Wajib dikuasai

Perintah	Fungsi
<code>docker-compose up --build</code>	Build image lalu jalankan semua service
<code>docker-compose up -d --build</code>	Jalankan di background (detached mode)
<code>docker-compose down</code>	Hentikan + hapus container. Volume aman.
<code>docker-compose down -v</code>	Hentikan + hapus container DAN volume. Data hilang.
<code>docker-compose logs -f</code>	Stream logs semua service secara real-time
<code>docker-compose logs -f risk-api</code>	Stream logs satu service spesifik
<code>docker-compose ps</code>	Lihat status semua service (Up/Exit/healthy)
<code>docker-compose stop</code>	Hentikan container tanpa menghapusnya
<code>docker-compose restart</code>	Restart semua service

7. Path Resolution yang Robust — config.py

Saat memindahkan `.env` ke root project, pattern `env_file = ".env"` di Pydantic bersifat relatif terhadap *current working directory* saat program dijalankan — bukan relatif terhadap lokasi file `config.py`. Ini menjadi sumber bug yang sulit dilacak. Solusi yang robust: gunakan `Path(__file__).resolve()` untuk mendapatkan path absolut.

```
from pathlib import Path
from pydantic_settings import BaseSettings

# __file__ = path absolut config.py ini sendiri
# .parent = folder app/
# .parent.parent = root project (tempat .env berada)
BASE_DIR = Path(__file__).resolve().parent.parent

class Settings(BaseSettings):
    MODEL_PATH: str = str(BASE_DIR / 'app/model/pipeline.joblib')
    APP_VERSION: str = '1.0.0'

    model_config = {
        'env_file': str(BASE_DIR / '.env'), # absolut, tidak peduli CWD
        'env_file_encoding': 'utf-8',
    }

settings = Settings()
```

■ class Config sudah deprecated di Pydantic v2. Gunakan `model_config = {}` sebagai dict.

■ Di dalam Docker container, `BASE_DIR` resolve ke `/app` karena `WORKDIR /app`. Model path menjadi `/app/app/model/...` — ini benar karena folder `app/` di-COPY ke `/app/`.

8. Docker Compose Default Network

Setiap file `docker-compose.yml` secara otomatis membuat satu **default bridge network** untuk semua service di dalamnya. Di dalam network ini, setiap service name berfungsi sebagai DNS hostname — Docker internal DNS menerjemahkan nama service ke IP container secara otomatis.

Implikasi penting: Dua file `docker-compose.yml` yang berbeda = dua network yang berbeda. Service di file A tidak bisa memanggil service di file B via nama service, meskipun keduanya berjalan bersamaan di host yang sama. Untuk menghubungkan dua compose project, diperlukan konfigurasi *external network* eksplisit.

9. Verifikasi — Log yang Sehat

Output log yang benar setelah `docker-compose up --build` berhasil:

```
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:app.main:[STARTUP] Risk Scoring API v1.0.0 starting...
INFO:app.services.model_service:Model successfully loaded from /app/app/model/...
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000
```

Request masuk — semua 200 OK

172.18.0.1:38294 - 'GET /docs HTTP/1.1' 200 OK

172.18.0.1:38294 - 'GET /health HTTP/1.1' 200 OK

■ `http://0.0.0.0:8000` bukan alamat yang bisa dibuka di browser. `0.0.0.0` artinya 'mendengarkan semua interface'. Akses via `http://localhost:8000`.

■ `172.18.0.1` adalah IP Docker bridge gateway — bukan IP eksternal. Normal.